AI-Assisted Development Exam

Family Medicine Register

A small local-first web application for recording family medicine purchases, viewing stock, and estimating when a package may run out.

Project idea and system requirements

The Family Medicine Register helps a household keep a separate medicine inventory for each family member. A user can create and remove profiles, add medicine purchases, find product details by barcode, and review estimated stock reminders in one browser interface.

- Family profiles and medicine records are stored in a local SQLite database and managed through a Flask web API.

- A barcode lookup checks Barcode.bg and public product catalogs for a product name, image, and package size; product details can still be entered manually.

- The interface displays the product image, barcode, purchase quantity/date, daily usage, and an estimated run-out reminder when units can be matched.

- Family names are unique without regard to case, including Cyrillic case variants. Removing a family profile also removes its associated medicine records.

Scope and safety

This is a local household record, not a medical dosing or prescribing system. Run-out dates are estimates derived from user-entered quantities, package size, daily usage, and purchase date. They do not track doses actually taken and should be checked against the real package and usage.

Technology and project reference

Python, Flask, SQLite, browser-native JavaScript, HTML, and CSS. Development used GitHub Copilot in VS Code with iterative prompts, code review, tests, and browser validation.

Repository:

https://github.com/sndi70/MedicineAI

Evidence and test results in this document were captured from the working application and its current test suite.

System architecture — modules

Browser UI ⇄ Flask JSON API ⇄ SQLite database ⇅ Barcode and product catalogs

The server validates input and owns persistence and inventory calculations. The browser renders profiles and medicine cards, calls the API, and presents lookup results and reminders. External catalog failures are surfaced rather than treated as reliable product data.

Module 1 — Flask API and validation

Approach & reasoning: Keep data rules on the server so browser forms and API clients receive consistent validation. Use explicit JSON routes for profiles, medicines, overview data, images, and barcode lookup.

Workflow: Asked Copilot to add the API behavior; inspected route and error handling; refined it with regression tests for invalid input, duplicate names, CRUD, and compatibility with existing routes.

Testing strategy: Flask's test client exercises successful and rejected requests, returned JSON shapes, HTTP status codes, and deletion behavior. Current full suite: 27 tests pass.

AI tool choice: GitHub Copilot in VS Code provided code-aware implementation suggestions and helped iterate against the existing Flask patterns.

Representative prompt: "Forbid creation of a new member with a name that already exists." Follow-up testing exposed SQLite NOCASE's ASCII-only behavior; the implementation now compares Python Unicode casefolded names.

Module 2 — SQLite persistence

Approach & reasoning: Keep the application local-first and dependency-light. SQLite stores family profiles and medicine purchases; profile deletion cascades to its medicines. The database is initialized automatically and lives under the ignored data/ directory.

Workflow: Asked Copilot to connect application routes to the existing SQLite schema; verified overview and CRUD responses; preserved the existing database filename so existing local data remains usable.

Testing strategy: Temporary databases isolate unit tests. Tests cover create/list/delete operations, profile-to-medicine relationships, and the combined overview endpoint.

AI tool choice: GitHub Copilot in VS Code was used to make small changes in the existing Flask/SQLite structure and generate focused regression tests.

Representative prompt: "Keep the medicine records separated by family member and allow a member to be removed."

Development process per module

Module 3 — Barcode lookup and product enrichment

Approach & reasoning: Barcode values identify candidates, not guaranteed medicine facts. Query Barcode.bg alongside Open Facts catalogs, prefer matching Barcode.bg product details where available, and retain fallback product images or package-size data from other catalogs. Allow manual entry when the lookup misses.

Workflow: Asked Copilot to add Barcode.bg, combine catalog responses, extract package counts from product names, and display a medicine image rather than its URL. Verified known barcode cases and the live lookup response.

Testing strategy: Mock catalog HTTP responses to test matching barcode rows, fallback data, network failures, and package-size extraction. The live Nebilet lookup returns “28 таблетки” for barcode 4013054002162.

AI tool choice: GitHub Copilot in VS Code supported implementation and test iteration. Browser review confirmed the populated image preview and package-size field.

Representative prompts: “Always check barcode in Barcode.bg.” “Look up package size automatically.”

Module 4 — Inventory estimates and reminders

Approach & reasoning: Calculate a calendar run-out date only when package quantity and daily usage can be interpreted in compatible units. Multiply package size by purchased package count, account for daily frequency, and start from purchase date. Display a persistent card warning when the estimated date is within three days or has passed.

Workflow: Asked Copilot for a three-day warning based on package size, daily usage, quantity, and buying date. Iteratively corrected bare-number inputs and package counts inferred from names, then checked visible UI warnings.

Testing strategy: Unit tests cover expected dates, frequency, unit mismatch, quantities, purchase dates, and package-size extraction. Browser review confirmed the warning is present on the corresponding medicine card.

AI tool choice: GitHub Copilot in VS Code helped trace the calculation from stored fields to API output and visible warning; tests were used to validate date math.

Representative prompt: “Add notification 3 days before medicine quantity ending based on quantity, daily usage and buying date.”

Modules, challenges, and tool comparison

Module 5 — Browser interface

Approach & reasoning: Use browser-native JavaScript and accessible HTML controls to keep the app small and easy to run. The interface renders family selection, medicine cards, barcode search, dialogs, image previews, warnings, and visible error messages.

Workflow: Asked Copilot to add the requested interaction; reloaded the running app and inspected actual content in the browser; adjusted behavior from user feedback (for example, previewing the product image and sorting stock by estimated run-out date).

Testing strategy: Manual browser checks verified profile selection, populated medicine cards, image display, search, form lookup, package-size value, and warnings. API and business rules are covered by unit tests.

AI tool choice: GitHub Copilot in VS Code was useful because code changes and browser/test feedback could be handled in the same development workflow.

Representative prompt: “When adding medicine, show the medicine picture instead of its address.”

Module 6 — Automated tests and verification

Approach & reasoning: Add regression tests around behavior that can silently fail: unit/date calculations, barcode parsing, duplicate Cyrillic names, and data relationships. Keep external network tests deterministic by mocking catalog responses.

Workflow: Asked Copilot to add regression tests alongside features; ran the full unittest suite after changes; used live browser/API checks for end-to-end behavior.

Testing strategy: `python -m unittest discover -s tests` currently reports 27 passing tests. `py_compile` and `git diff --check` were also used after Python changes.

AI tool choice: GitHub Copilot in VS Code accelerated test scaffolding; the Python unittest runner provided the pass/fail evidence.

Representative interaction: “Add tests for exact and case-differing duplicate family names.”

Key prompts or interactions

Selected prompts, lightly edited for readability:

- “Add a persistent warning three days before medicine stock runs out, based on package size, daily usage, quantity, and purchase date.”
- “Always check Barcode.bg when searching by barcode, while keeping other catalogs as fallbacks.”
- “Look up the package size automatically, and show the medicine picture instead of its URL in the add form.”
- “Prevent creating a family member whose name matches an existing member, including capitalization variants.”
- “Add regression tests for exact and case-differing duplicate names.”

Challenges & tool comparison

The hardest issue was inconsistent catalog data: package counts may appear as “28 бр.”, tablet counts may be embedded in a product name, and external services can be unavailable. The UI therefore permits manual entry and does not assume every lookup is complete. Another challenge was Unicode-aware uniqueness: SQLite NOCASE did not equate Bulgarian Cyrillic case variants, so server-side Unicode casefold comparison was needed.

GitHub Copilot was the most useful tool in this project because it was integrated with the VS Code workspace and supported an iterative loop of implementation, tests, and browser inspection. I did not perform a controlled comparison with Claude Code, Augment Code, or Cursor, so this is a project-specific assessment rather than a benchmark.

If continuing: add encrypted backup/export and restore, track actual doses or stock adjustments, broaden tests for catalog edge cases, and add authentication before any remote or multi-user deployment. Reminder dates remain informational estimates.

Working system evidence

The following screenshots were captured from the running local application on 30 October 2026. Sample profile labels are abbreviated in the test data.

Figure 1. Medicine inventory with product images and an estimated run-out warning.

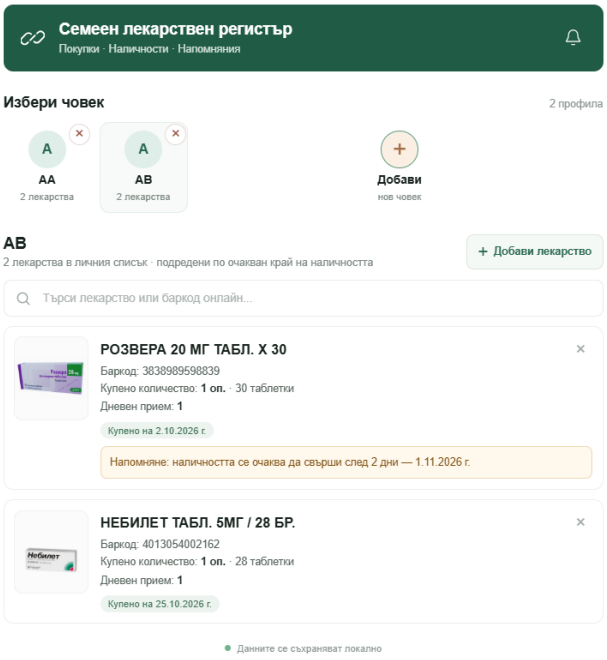
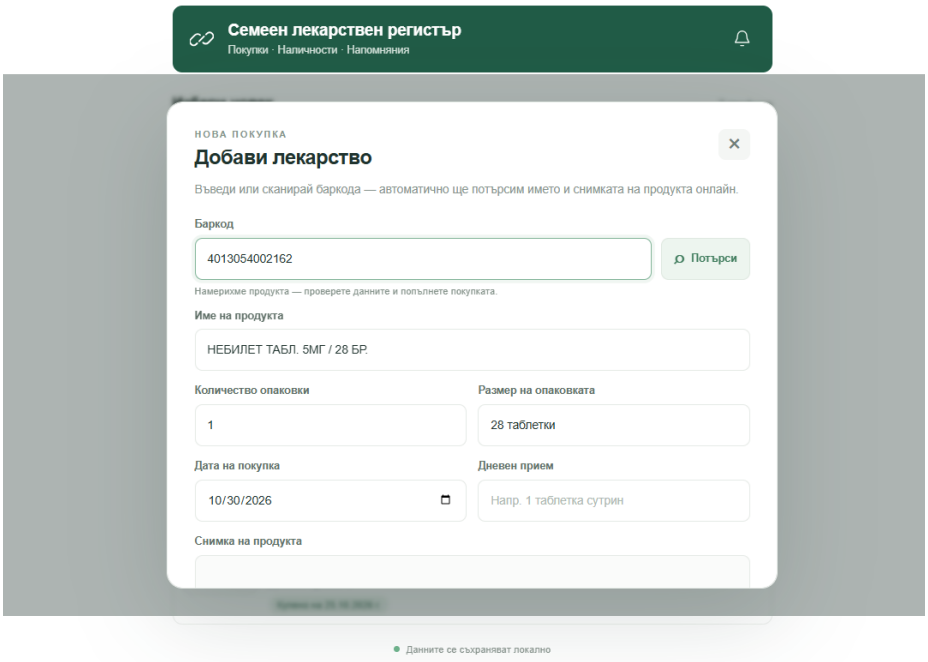


Figure 2. Barcode 4013054002162 fills the product name and package count (“28 таблетки”) in the add-medicine form.



Repository and verification

Source code: https://github.com/sndi70/MedicineAI

Verified test result: 27 tests passed using the project’s unittest suite. The live barcode lookup endpoint returned the Nebilet product and “28 таблетки”; the browser showed the corresponding populated form.